

Experiment 7

1. Aim

Implement secure error handling and logging mechanism using CERT/MISRA.

2. Hardware / Software Required

- **Operating System:** Windows/Linux
- **Python 3.x** / C/C++ compiler
- **VS Code** or any suitable IDE
- **GCC/G++** compiler (for C/C++ implementation)
- Logging facility such as Python logging or system logging
- Optional: **SonarQube** / static-analysis tool for checking coding-rule violations

3. Theory

Secure error handling and logging is the practice of detecting, handling, and recording errors in a controlled manner without exposing sensitive information or causing unexpected application behavior.

CERT (Computer Emergency Response Team) secure coding standards provide guidelines for writing safer software and preventing common programming errors such as improper input validation, resource leaks, integer errors, and improper exception handling.

MISRA (Motor Industry Software Reliability Association) guidelines are coding standards primarily designed for **C and C++**, especially in safety-critical and embedded systems. MISRA emphasizes predictable program behavior, defensive programming, controlled error handling, and avoidance of unsafe language features.

Procedure

1. Create a secure application with functions that may generate errors, such as invalid input, file errors, or division by zero.
2. Validate all inputs before processing them.
3. Handle errors using controlled mechanisms such as try-except or proper error-return codes instead of exposing system details.
4. Log important security events, including:
 - Invalid input
 - Authentication failures
 - Access violations
 - Application errors
5. Avoid logging sensitive information such as passwords, API keys, tokens, and personal data.

6. Use meaningful log levels such as INFO, WARNING, and ERROR.
7. Prevent information leakage by displaying a generic error message to the user while keeping technical details in the log.
8. Follow CERT/MISRA principles such as defensive programming, proper error handling, predictable control flow, and avoiding unsafe constructs.
9. Test the application with valid and invalid inputs and verify that errors are handled safely and recorded correctly.

CERT & MISRA Compliance Matrix

Rule / Guideline	Requirement	How It Is Satisfied
MISRA C:2012 Rule 17.7	Do not discard function return values.	Return codes from Secure_LogEvent are captured; calls to fclose/fflush are explicitly cast to (void).
CERT STR30-C	Ensure strings are null-terminated.	Custom Secure_StringCopy explicitly forces null-termination at index max_len - 1.
MISRA C:2012 Rule 21.6	Standard I/O library shall not be used unsafely.	Logging avoids raw stack-based string interpolation (sprintf); bounds are enforced via defined array sizes.
CERT ERR33-C	Detect standard library errors.	Return status from fopen and fprintf is evaluated before proceeding.
CWE-532 / Security	Avoid logging sensitive details.	Data buffer contents are excluded from log statements; only operational metadata and sanitized status text are recorded.

Python Code

```

1  """
2  Secure Error Handling and Logging Application
3  =====
4  Implements CERT/MISRA principles:
5      - Defensive programming with input validation
6      - Controlled error handling (try-except, no raw tracebacks to user)
7      - Security-event logging (invalid input, auth failures, access violations)
8      - Sensitive-data redaction in logs
9      - Meaningful log levels (INFO, WARNING, ERROR)
10     - Generic user-facing messages; technical details only in logs
11 """
12
13 import logging
14 import os
15 import re
16 import hashlib
17 import json
18 from datetime import datetime
19
20 # -----
21 # 1. SECURE LOGGER SETUP
22 # -----
23
24 LOG_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)),
25 "security_events.log")
26
27 # Sensitive patterns to redact before writing to any log
28 _SENSITIVE_PATTERNS = [
29     (re.compile(r'(password\s*[:]\s*)\S+', re.IGNORECASE), r'\1[REDACTED]'),
30     (re.compile(r'(api[_-]?key\s*[:]\s*)\S+', re.IGNORECASE), r'\1[REDACTED]'),
31     (re.compile(r'(token\s*[:]\s*)\S+', re.IGNORECASE), r'\1[REDACTED]'),
32     (re.compile(r'(ssn\s*[:]\s*)\S+', re.IGNORECASE), r'\1[REDACTED]'),
33     (re.compile(r'(credit[_-]?card\s*[:]\s*)\S+', re.IGNORECASE), r'\1[REDACTED]'),
34 ]
35
36 class SanitizingFormatter(logging.Formatter):
37     """Custom formatter that strips sensitive data from every log record."""
38
39     def format(self, record):
40         message = super().format(record)
41         for pattern, replacement in _SENSITIVE_PATTERNS:
42             message = pattern.sub(replacement, message)
43         return message
44
45
46 def _build_logger() -> logging.Logger:
47     """Create and return the application-wide secure logger."""
48     logger = logging.getLogger("SecureApp")
49     logger.setLevel(logging.DEBUG)

```

```

50
51     formatter = SanitizingFormatter(
52         fmt="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
53         datefmt="%Y-%m-%d %H:%M:%S",
54     )
55
56     # File handler - detailed logs (DEBUG and above)
57     fh = logging.FileHandler(LOG_FILE, mode="w", encoding="utf-8")
58     fh.setLevel(logging.DEBUG)
59     fh.setFormatter(formatter)
60
61     # Console handler - only WARNING and above (no technical spill to users)
62     ch = logging.StreamHandler()
63     ch.setLevel(logging.WARNING)
64     ch.setFormatter(formatter)
65
66     logger.addHandler(fh)
67     logger.addHandler(ch)
68     return logger
69
70
71 logger = _build_logger()
72
73 # -----
74 # 2. CUSTOM EXCEPTION HIERARCHY (CERT ERR06-J / defensive layering)
75 # -----
76
77
78 class AppError(Exception):
79     """Base application error - never expose internals to user."""
80
81     def __init__(self, user_message: str, internal_detail: str = ""):
82         super().__init__(user_message)
83         self.user_message = user_message
84         self.internal_detail = internal_detail
85
86
87 class ValidationError(AppError):
88     """Raised when user-supplied input fails validation."""
89
90
91 class AuthenticationError(AppError):
92     """Raised on authentication failure."""
93
94
95 class AccessViolationError(AppError):
96     """Raised when an authenticated user lacks permission."""
97
98
99 class FileOperationError(AppError):
100     """Raised on secure file-operation failures."""
101
102
103 # -----

```

```

104 # 3. INPUT VALIDATION (CERT IDS00, defensive programming)
105 # -----
106
107
108 def validate_username(username: str) -> str:
109     """Allow only 3-20 alphanumeric / underscore characters."""
110     if not isinstance(username, str):
111         raise ValidationError(
112             "Invalid input provided.",
113             f"Username type error: expected str, got {type(username).__name__}",
114         )
115     username = username.strip()
116     if not re.fullmatch(r'[A-Za-z0-9_]{3,20}', username):
117         raise ValidationError(
118             "Invalid input provided.",
119             f"Username validation failed for value length={len(username)}",
120         )
121     return username
122
123
124 def validate_positive_number(value, label: str = "number") -> float:
125     """Ensure *value* is a finite positive number."""
126     try:
127         num = float(value)
128     except (TypeError, ValueError) as exc:
129         raise ValidationError(
130             "Invalid input provided.",
131             f"{label} conversion failed: {exc}",
132         )
133     if num != num: # NaN check (CERT: avoid undefined behaviour)
134         raise ValidationError(
135             "Invalid input provided.",
136             f"{label} is NaN",
137         )
138     if num <= 0:
139         raise ValidationError(
140             "Invalid input provided.",
141             f"{label} must be positive, got {num}",
142         )
143     return num
144
145
146 def validate_file_path(path: str) -> str:
147     """Reject path traversal and non-string paths (CERT FI016)."""
148     if not isinstance(path, str) or not path.strip():
149         raise ValidationError(
150             "Invalid input provided.",
151             "File path is empty or not a string.",
152         )
153     # Block directory traversal attempts
154     normalised = os.path.normpath(path)
155     if ".." in normalised.split(os.sep):
156         raise ValidationError(
157             "Invalid input provided.",

```

```
158         f"Path traversal detected in: {path}",
159     )
160     return normalised
161
162
163     # -----
164     # 4. AUTHENTICATION SIMULATION
165     # -----
166
167     # Pre-stored hashed credentials (never store plain-text passwords)
168     _USER_DB: dict[str, dict] = {
169         "admin": {
170             "password_hash": hashlib.sha256("Str0ng!Pass".encode()).hexdigest(),
171             "role": "admin",
172         },
173         "viewer": {
174             "password_hash": hashlib.sha256("View@2024".encode()).hexdigest(),
175             "role": "viewer",
176         },
177     }
178
179
180     def authenticate(username: str, password: str) -> dict:
181         """
182         Authenticate a user. Returns user record on success.
183         Logs failure WITHOUT echoing the password.
184         """
185         username = validate_username(username)
186
187         user = _USER_DB.get(username)
188         if user is None:
189             logger.warning("AUTH_FAILURE | Unknown username attempted login. username=%s",
190                             username)
191             raise AuthenticationError(
192                 "Authentication failed. Please check your credentials.",
193                 f"No such user: {username}",
194             )
195
196         pwd_hash = hashlib.sha256(password.encode()).hexdigest()
197         if pwd_hash != user["password_hash"]:
198             # CERT: never log the actual password
199             logger.warning(
200                 "AUTH_FAILURE | Incorrect password for user=%s (password=[REDACTED])",
201                 username
202             )
203             raise AuthenticationError(
204                 "Authentication failed. Please check your credentials.",
205                 f"Password mismatch for user: {username}",
206             )
207
208         logger.info("AUTH_SUCCESS | User '%s' authenticated successfully (role=%s).",
209                     username, user["role"])
210         return {"username": username, "role": user["role"]}
```

```
209
210 # -----
211 # 5. ACCESS CONTROL
212 # -----
213
214
215 def authorize(user: dict, required_role: str) -> None:
216     """Check role-based access. Raise AccessViolationError if denied."""
217     role_hierarchy = {"admin": 2, "viewer": 1}
218     user_level = role_hierarchy.get(user.get("role"), 0)
219     required_level = role_hierarchy.get(required_role, 99)
220
221     if user_level < required_level:
222         logger.error(
223             "ACCESS_VIOLATION | User '%s' (role=%s) attempted action requiring '%s'.",
224             user["username"], user["role"], required_role,
225         )
226         raise AccessViolationError(
227             "You do not have permission to perform this action.",
228             f"User {user['username']} denied; needs role={required_role}",
229         )
230     logger.info("ACCESS_GRANTED | User '%s' authorized for role '%s'.",
231                user["username"], required_role)
232
233 # -----
234 # 6. BUSINESS OPERATIONS (may raise controlled errors)
235 # -----
236
237
238 def safe_divide(numerator, denominator) -> float:
239     """Division with full input validation (CERT INT33-C / FLP03)."""
240     num = validate_positive_number(numerator, "numerator")
241     den = validate_positive_number(denominator, "denominator")
242
243     # Redundant guard - defensive programming
244     if den == 0:
245         logger.error("DIVISION_BY_ZERO | Attempt to divide %s by zero.", num)
246         raise AppError(
247             "A calculation error occurred. Please try again.",
248             "Division by zero after validation (should not happen).",
249         )
250
251     result = num / den
252     logger.info("CALC_SUCCESS | %s / %s = %s", num, den, result)
253     return result
254
255
256 def read_file_securely(path: str) -> str:
257     """Read a file with path validation and controlled error handling."""
258     safe_path = validate_file_path(path)
259
260     if not os.path.isfile(safe_path):
261         logger.error("FILE_ERROR | File not found: %s", safe_path)
```

```
262         raise FileNotFoundError(
263             "The requested file could not be found.",
264             f"Missing file: {safe_path}",
265         )
266
267     try:
268         with open(safe_path, "r", encoding="utf-8") as fh:
269             content = fh.read()
270             logger.info("FILE_READ | Successfully read file: %s (%d bytes)", safe_path,
271 len(content))
272         return content
273     except PermissionError:
274         logger.error("FILE_ERROR | Permission denied reading: %s", safe_path)
275         raise FileNotFoundError(
276             "Unable to access the requested file.",
277             f"Permission denied: {safe_path}",
278         )
279     except OSError as exc:
280         logger.error("FILE_ERROR | OS error reading %s: %s", safe_path, exc)
281         raise FileNotFoundError(
282             "An error occurred while reading the file.",
283             f"OSError: {exc}",
284         )
285
286 def process_user_data(data: str) -> dict:
287     """Parse JSON user data with sensitive-field redaction in logs."""
288     if not isinstance(data, str) or not data.strip():
289         logger.warning("INVALID_INPUT | Empty or non-string data received for
290 processing.")
291         raise ValidationError(
292             "Invalid input provided.",
293             "process_user_data received empty/non-string input",
294         )
295
296     try:
297         parsed = json.loads(data)
298     except json.JSONDecodeError as exc:
299         logger.warning("INVALID_INPUT | Malformed JSON: %s", exc)
300         raise ValidationError(
301             "Invalid input provided.",
302             f"JSON decode error: {exc}",
303         )
304
305     if not isinstance(parsed, dict):
306         logger.warning("INVALID_INPUT | JSON is not an object.")
307         raise ValidationError(
308             "Invalid input provided.",
309             "Expected JSON object, got " + type(parsed).__name__,
310         )
311
312     # Redact sensitive fields before any logging (CERT: data minimisation)
313     safe_log = {
```



```

313         k: ("[REDACTED]" if k.lower() in {"password", "ssn", "credit_card", "token",
"api_key"} else v)
314         for k, v in parsed.items()
315     }
316     logger.info("DATA_PROCESSED | Received user data: %s", safe_log)
317     return parsed
318
319
320 # -----
321 # 7. TOP-LEVEL ERROR BOUNDARY (generic message to user)
322 # -----
323
324
325 def safe_execute(label: str, func, *args, **kwargs):
326     """
327     Execute *func* inside a controlled error boundary.
328     • AppError → log detail, show user_message
329     • Any other → log full detail, show only generic message
330     """
331     print(f"\n{'='*60}")
332     print(f"  TEST: {label}")
333     print(f"{'='*60}")
334     try:
335         result = func(*args, **kwargs)
336         print(f"  [OK] Result: {result}")
337         return result
338     except AppError as exc:
339         # Controlled application error
340         logger.error("APP_ERROR | %s | internal: %s", exc.user_message,
exc.internal_detail)
341         print(f"  [FAIL] {exc.user_message}") # generic to user
342     except Exception as exc:
343         # Unexpected error – never expose traceback to user (CERT ERR01)
344         logger.critical(
345             "UNEXPECTED_ERROR | %s | %s: %s",
346             label, type(exc).__name__, exc,
347             exc_info=True,
348         )
349         print("  [FAIL] An unexpected error occurred. Please contact support.")
350
351
352 # -----
353 # 8. TEST SUITE – valid & invalid inputs
354 # -----
355
356
357 def run_tests() -> None:
358     """Exercise every error-handling and logging path."""
359
360     print("\n" + "=" * 60)
361     print("  SECURE ERROR HANDLING & LOGGING -- CERT/MISRA DEMO")
362     print("=" * 60)
363
364     # -- Authentication tests -----

```

```

365     safe_execute(
366         "Auth - valid admin login",
367         authenticate, "admin", "Str0ng!Pass",
368     )
369     safe_execute(
370         "Auth - wrong password (should fail gracefully)",
371         authenticate, "admin", "wrong_password",
372     )
373     safe_execute(
374         "Auth - unknown user",
375         authenticate, "hacker", "anything",
376     )
377     safe_execute(
378         "Auth - invalid username format (SQL-injection attempt)",
379         authenticate, "'; DROP TABLE--", "pwd",
380     )
381
382     # -- Access-control tests -----
383     viewer = {"username": "viewer", "role": "viewer"}
384     admin = {"username": "admin", "role": "admin"}
385
386     safe_execute(
387         "Access - viewer requests admin action (should be denied)",
388         authorize, viewer, "admin",
389     )
390     safe_execute(
391         "Access - admin requests admin action (should succeed)",
392         authorize, admin, "admin",
393     )
394
395     # -- Division tests -----
396     safe_execute("Divide - valid: 100 / 3", safe_divide, 100, 3)
397     safe_execute("Divide - zero denominator", safe_divide, 10, 0)
398     safe_execute("Divide - negative number", safe_divide, -5, 2)
399     safe_execute("Divide - non-numeric input", safe_divide, "abc", 2)
400
401     # -- File-read tests -----
402     safe_execute("File - read non-existent file", read_file_securely,
403 "no_such_file.txt")
404     safe_execute("File - path traversal attack", read_file_securely,
405 "../../etc/passwd")
406
407     # Create a temp file to test successful read
408     tmp_path = os.path.join(os.path.dirname(os.path.abspath(__file__)),
409 "_test_sample.txt")
410     try:
411         with open(tmp_path, "w", encoding="utf-8") as f:
412             f.write("Hello, secure world!")
413         safe_execute("File - read valid file", read_file_securely, tmp_path)
414     finally:
415         if os.path.exists(tmp_path):
416             os.remove(tmp_path)
417
418     # -- JSON processing tests -----

```

```
416     safe_execute(  
417         "JSON - valid data with sensitive fields (should be redacted in log)",  
418         process_user_data,  
419         '{"name": "Alice", "email": "alice@example.com", "password": "s3cret!", "ssn":  
"123-45-6789"}',  
420     )  
421     safe_execute("JSON - malformed JSON", process_user_data, "{bad json}")  
422     safe_execute("JSON - empty string", process_user_data, "")  
423     safe_execute("JSON - array instead of object", process_user_data, "[1,2,3]")  
424  
425     # -- Unexpected error simulation -----  
426     def cause_unexpected():  
427         return {"a": 1}["missing_key"]  
428  
429     safe_execute("Unexpected - KeyError (unhandled type)", cause_unexpected)  
430  
431     # -- Summary -----  
432     print("\n" + "=" * 60)  
433     print(" [LOG] Detailed security log written to:")  
434     print(f"         {LOG_FILE}")  
435     print("=" * 60)  
436  
437  
438     # -----  
439     # 9. ENTRY POINT  
440     # -----  
441  
442     if __name__ == "__main__":  
443         run_tests()  
444
```

security_events.log

2026-08-31 12:34:40 | INFO | SecureApp | AUTH_SUCCESS | User 'admin' authenticated successfully (role=admin).

2026-08-31 12:34:40 | WARNING | SecureApp | AUTH_FAILURE | Incorrect password for user=admin (password=[REDACTED])

2026-08-31 12:34:40 | ERROR | SecureApp | APP_ERROR | Authentication failed. Please check your credentials. | internal: Password mismatch for user: admin

2026-08-31 12:34:40 | WARNING | SecureApp | AUTH_FAILURE | Unknown username attempted login. username=hacker

2026-08-31 12:34:40 | ERROR | SecureApp | APP_ERROR | Authentication failed. Please check your credentials. | internal: No such user: hacker

2026-08-31 12:34:40 | ERROR | SecureApp | APP_ERROR | Invalid input provided. | internal: Username validation failed for value length=15

2026-08-31 12:34:40 | ERROR | SecureApp | ACCESS_VIOLATION | User 'viewer' (role=viewer) attempted action requiring 'admin'.

2026-08-31 12:34:40 | ERROR | SecureApp | APP_ERROR | You do not have permission to perform this action. | internal: User viewer denied; needs role=admin

2026-08-31 12:34:40 | INFO | SecureApp | ACCESS_GRANTED | User 'admin' authorized for role 'admin'.

2026-08-31 12:34:40 | INFO | SecureApp | CALC_SUCCESS | $100.0 / 3.0 = 33.333333333333336$

2026-08-31 12:34:40 | ERROR | SecureApp | APP_ERROR | Invalid input provided. | internal: denominator must be positive, got 0.0

2026-08-31 12:34:40 | ERROR | SecureApp | APP_ERROR | Invalid input provided. | internal: numerator must be positive, got -5.0

2026-08-31 12:34:40 | ERROR | SecureApp | APP_ERROR | Invalid input provided. | internal: numerator conversion failed: could not convert string to float: 'abc'

2026-08-31 12:34:40 | ERROR | SecureApp | FILE_ERROR | File not found: no_such_file.txt

2026-08-31 12:34:40 | ERROR | SecureApp | APP_ERROR | The requested file could not be found. | internal: Missing file: no_such_file.txt

2026-08-31 12:34:40 | ERROR | SecureApp | APP_ERROR | Invalid input provided. | internal: Path traversal detected in: ../../etc/passwd

2026-08-31 12:34:40 | INFO | SecureApp | FILE_READ | Successfully read file:
d:\OneDrive\Desktop_test_sample.txt (20 bytes)

2026-08-31 12:34:40 | INFO | SecureApp | DATA_PROCESSED | Received user data: {'name': 'Alice',
'email': 'alice@example.com', 'password': '[REDACTED]', 'ssn': '[REDACTED]'}

2026-08-31 12:34:40 | WARNING | SecureApp | INVALID_INPUT | Malformed JSON: Expecting
property name enclosed in double quotes: line 1 column 2 (char 1)

2026-08-31 12:34:40 | ERROR | SecureApp | APP_ERROR | Invalid input provided. | internal: JSON
decode error: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)

2026-08-31 12:34:40 | WARNING | SecureApp | INVALID_INPUT | Empty or non-string data received
for processing.

2026-08-31 12:34:40 | ERROR | SecureApp | APP_ERROR | Invalid input provided. | internal:
process_user_data received empty/non-string input

2026-08-31 12:34:40 | WARNING | SecureApp | INVALID_INPUT | JSON is not an object.

2026-08-31 12:34:40 | ERROR | SecureApp | APP_ERROR | Invalid input provided. | internal: Expected
JSON object, got list

2026-08-31 12:34:40 | CRITICAL | SecureApp | UNEXPECTED_ERROR | Unexpected - KeyError
(unhandled type) | KeyError: 'missing_key'

Traceback (most recent call last):

File "d:\OneDrive\Desktop\secure_app.py", line 335, in safe_execute

result = func(*args, **kwargs)

File "d:\OneDrive\Desktop\secure_app.py", line 427, in cause_unexpected

return {"a": 1}["missing_key"]

~~~~~^

KeyError: 'missing\_key'

## **Output Analysis**

```
python -u "d:\OneDrive\Desktop\secure_app.py"
```

```
=====
SECURE ERROR HANDLING & LOGGING -- CERT/MISRA DEMO
=====
```

```
=====
TEST: Auth - valid admin login
=====
```

```
[OK] Result: {'username': 'admin', 'role': 'admin'}
```

```
=====
TEST: Auth - wrong password (should fail gracefully)
=====
```

```
2026-08-31 12:45:58 | WARNING | SecureApp | AUTH_FAILURE | Incorrect password for user=admin (password=[REDACTED])
2026-08-31 12:45:58 | ERROR | SecureApp | APP_ERROR | Authentication failed. Please check your credentials. | internal: Password mismatch for user: admin
[FAIL] Authentication failed. Please check your credentials.
```

```
=====
TEST: Auth - unknown user
=====
```

```
2026-08-31 12:45:58 | WARNING | SecureApp | AUTH_FAILURE | Unknown username attempted login. username=hacker
2026-08-31 12:45:58 | ERROR | SecureApp | APP_ERROR | Authentication failed. Please check your credentials. | internal: No such user: hacker
[FAIL] Authentication failed. Please check your credentials.
```

```
=====
TEST: Auth - invalid username format (SQL-injection attempt)
=====
```

```
2026-08-31 12:45:58 | ERROR | SecureApp | APP_ERROR | Invalid input provided. | internal: Username validation failed for value length=15
[FAIL] Invalid input provided.
```

```
=====
TEST: Access - viewer requests admin action (should be denied)
=====
```

```
=====
TEST: Access - viewer requests admin action (should be denied)
=====
```

```
2026-08-31 12:45:58 | ERROR | SecureApp | ACCESS_VIOLATION | User 'viewer' (role=viewer) attempted action requiring 'admin'.
2026-08-31 12:45:58 | ERROR | SecureApp | APP_ERROR | You do not have permission to perform this action. | internal: User viewer denied; needs role=admin
[FAIL] You do not have permission to perform this action.
```

```
=====
TEST: Access - admin requests admin action (should succeed)
=====
```

```
[OK] Result: None
```

```
=====
TEST: Divide - valid: 100 / 3
=====
```

```
[OK] Result: 33.333333333333336
```

```
=====
TEST: Divide - zero denominator
=====
```

```
2026-08-31 12:45:58 | ERROR | SecureApp | APP_ERROR | Invalid input provided. | internal: denominator must be positive, got 0.0
[FAIL] Invalid input provided.
```

```
=====
TEST: Divide - negative number
=====
```

```
2026-08-31 12:45:58 | ERROR | SecureApp | APP_ERROR | Invalid input provided. | internal: numerator must be positive, got -5.0
[FAIL] Invalid input provided.
```

```
=====
TEST: Divide - non-numeric input
=====
```

```
2026-08-31 12:45:58 | ERROR | SecureApp | APP_ERROR | Invalid input provided. | internal: numerator conversion failed: could not convert string to float: 'abc'
```

```

-----
2026-08-31 12:45:58 | ERROR    | SecureApp | APP_ERROR | Invalid input provided. | internal: numerator conversion failed: could not convert string to float: 'abc'
[FAIL] Invalid input provided.

=====
TEST: File - read non-existent file
=====
2026-08-31 12:45:58 | ERROR    | SecureApp | FILE_ERROR | File not found: no_such_file.txt
2026-08-31 12:45:58 | ERROR    | SecureApp | APP_ERROR | The requested file could not be found. | internal: Missing file: no_such_file.txt
[FAIL] The requested file could not be found.

=====
TEST: File - path traversal attack
=====
2026-08-31 12:45:58 | ERROR    | SecureApp | APP_ERROR | Invalid input provided. | internal: Path traversal detected in: ../../etc/passwd
[FAIL] Invalid input provided.

=====
TEST: File - read valid file
=====
[OK] Result: Hello, secure world!

=====
TEST: JSON - valid data with sensitive fields (should be redacted in log)
=====
[OK] Result: {'name': 'Alice', 'email': 'alice@example.com', 'password': 's3cret!', 'ssn': '123-45-6789'}

=====
TEST: JSON - malformed JSON
=====
2026-08-31 12:45:58 | WARNING  | SecureApp | INVALID_INPUT | Malformed JSON: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)
2026-08-31 12:45:58 | ERROR    | SecureApp | APP_ERROR | Invalid input provided. | internal: JSON decode error: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)
[FAIL] Invalid input provided.

```

## Additional Learning

1. Learned the importance of **secure error handling** in preventing application crashes and unexpected behavior.
2. Understood how **CERT secure coding guidelines** help prevent common programming and security vulnerabilities.
3. Learned the basic principles of **MISRA coding standards**, especially for developing reliable and predictable C/C++ programs.
4. Understood how to use **logging mechanisms** to record application errors and security-related events.
5. Learned to use appropriate **log levels** such as INFO, WARNING, ERROR, and CRITICAL.
6. Understood why **sensitive information such as passwords, tokens, and keys should not be logged**.
7. Learned how generic error messages can prevent **information leakage** while detailed information is safely stored in logs.
8. Understood the role of **input validation and defensive programming** in secure software development.
9. Learned that proper error handling improves **software reliability, maintainability, and security**.

## Conclusion

Thus, a secure error-handling and logging mechanism was successfully implemented following CERT/MISRA secure coding principles. The implementation demonstrated proper input validation, controlled error handling, secure logging, and prevention of sensitive information disclosure. These practices help improve the reliability, security, and maintainability of software applications.